Task Allocation via Verifiable Market Games

Riyaz Ahuja

March 26, 2026

Mellon College of Science
Department of Mathematics
Carnegie Mellon University
Pittsburgh, Pennsylvania 15213

Thesis Committee:

Dr. Prasad Tetali, Chair
Dr. Jeremy Avigad
Dr. Sean Welleck

Thesis submitted in partial fulfillment of the requirements for the degree of Master of Science in Mathematics

© Riyaz Ahuja, 2026

Dedicated to todo

Acknowledgments

todo

Abstract

Mathematical discovery is not a sequence of isolated proofs, but a distributed process across mathematicians and their expertise. We therefore model mathematical development as a scheduling and resource allocation problem, formalizing it as the Interdependent Task Allocation Problem (ITAP). Within a centralized-control regime, we study ITAP's complexity and approximation-theoretic structure. However, when agent capabilities are private, this regime becomes fragile, suggesting a decentralized formulation in which coordination is strategic rather than purely combinatorial is more tractible. We formalize this as a Verifiable Allocation Market POSG (VAMP) and show that exact equilibria computation is intractible in general, motivating a multi-agent reinforcement learning framework that approximates no-regret behavior in the empirical meta-game. We evaluate this model on synthetic ITAP environments against centralized baselines and apply the framework for theorem proving in Lean, showing that it improves multi-agent strategy amongst LLM baselines on research-level theorem proving tasks.

Table of Contents

List of Figures vi

List of Tables vii

1 **Introduction** 1

1.1 Motivation 1

1.2 Core Contributions 3

2 **Centralized Multi-Agent Theorem Proving** 5

2.1 Background 5

2.2 The Centralized MATP Problem 8

2.3 MATP and Scheduling Theory 17

2.4 Complexity and Approximability 21

3 **Decentralized Multi-Agent Theorem Proving** 31

3.1 Background: Multi-Agent Decision Making 31

3.2 The Decentralized MATP Game 32

3.3 The Publication Externality 34

3.4 Market-Mediated MATP 36

3.5 Internalization of Externalities 37

3.6 Equilibrium Analysis 38

3.7 Intractibility Consequences 40

4 **Multiagent Reinforcement Learning for VAMPs** 41

5 **Experiments** 42

6 **Application to Neural Theorem Proving** 43

7 **Conclusion** 44

Bibliography 45

List of Figures

Figure 1.1 Schematic dependency graph for the Lean formalization effort around Fermat's Last Theorem. The intent is to visualize a theorem not as an isolated statement, but as the endpoint of a large directed dependency structure spanning multiple mathematical domains. Such a graph highlights why theorem proving, formal verification, and related activities may be viewed as allocation problems over interdependent tasks. 2

List of Tables

Table 2.1 Complexity and approximation landscape for Non-Conjecturing PPI MATP. "Latent δ^*" indicates the general case with unique but unknown dependencies. "LPT" is Longest Processing Time first; "RLS" is Reactive List Scheduling (Thm. 2.5); "LST" is Lenstra–Shmoys–Tardos LP rounding; "str." abbreviates "strongly." All approximation ratios are with respect to the expected makespan objective. 29

Chapter 1

Introduction

1.1 Motivation

In 1637, Pierre de Fermat famously conjectured that for $n > 2$, there are no positive integer solutions to the equation $x^n + y^n = z^n$. Despite Fermat's claim to have a proof of this statement, his textbook margins provided too little space for his supposed proof, and thus this conjecture remained unproven for 358 years. It was only in 1994 that Andrew Wiles, building on decades of work by many mathematicians across domains, finally proved Fermat's Last Theorem.

Wiles' proof of Fermat's Last Theorem did not arise from a direct attack on the original equation. Instead, it emerged from deep developments and decades of study of domains seemingly - at first glance - far removed from the scope of the original conjecture. Throughout the 20th century, the theory of elliptic curves and modular forms was developed extensively, not only for the purposes of resolving FLT, but rather for a more fundamental understanding of the connections between arithmetic and geometric objects (elliptic curves over $\mathbb{Q}$) with complex analysis. Indeed, even Wiles' own proof of FLT was but a corrolary of his proof of the deeper Taniyama-Shimura conjecture, which was developed as a central question of the unification of large branches of mathematics. And yet, without these foundational contributions to the theory – motivated not by the resolution of a particular conjecture but rather in search of a deeper understanding of the mathematical landscape – the proof of FLT would not have been conceivable, as the language to discuss it would not exist.

TODO POLISH

[Now discuss how this pattern is common in mathematics, and how it is the fundamental collaborative, community efforts that lead to these large breakthroughs. Looking at the dependency graph for the Lean formalization of FLT, we see that the proof of a single target theorem may require the coordinated development of many supporting results across multiple domains that did not exist at the time of conjecture [fig 1]. And to a certain extent, this

Placeholder for dependency graph of the Lean FLT project.

Figure 1.1. Schematic dependency graph for the Lean formalization effort around Fermat's Last Theorem. The intent is to visualize a theorem not as an isolated statement, but as the endpoint of a large directed dependency structure spanning multiple mathematical domains. Such a graph highlights why theorem proving, formal verification, and related activities may be viewed as allocation problems over interdependent tasks.

pattern of corpus creation directly leading to major breakthroughs is common even beyond mathematics, as these corpora provide the infrastructure and language that shape the way we think about problems and their solutions. [Like the meme about god being a shepherd, pilot, author, or computer scientist]]

[Now, with the acceration of computational tooling and the rise of machine learning, we are seeing the rapid emergence of new tools for trustworthy automation and scaled collaboration in mathematics, and the integration of these tools through agentic AI systems. These systems are increasingly being used to tackle large, interdependent bodies of mathematical work by high profile scientists, and are being deployed at scale for the formalization of research-level mathematics. However, these systems are still in their infancy, viewing mathematical development as a search problem rather than an allocation problem.]

[Towards the advancement of such digitized mathemtical ecosystems, we interpret the problem of mathematical development as an instance of a more general problem of allocating heterogeneous, distributed agents over a large pool of interdependent tasks: proving, conjecturing, collaborating, or even competing. Namely, this thesis states and studies this interdependent task allocation problem, both in centralized and decentralized regimes, and develops a game-theoretic framework for the latter in which coordination is mediated by a market mechanism. We study equilibria in this game, and develop learning-based methods for approximating weak equilibria behavior in large instances. We implement and evaluate these methods on synthetic environments, and we instantiate the framework in neural theorem proving within Lean, studying its performance on research-level theorem proving tasks against agentic LLM baselines.]

1.2 Core Contributions

The main contributions of this thesis are as follows.

1. We formalize the *Interdependent Task Allocation Problem* (ITAP), a framework for allocating heterogeneous agents over tasks linked by logical or operational dependencies, with support for recursive task generation.

2. We distinguish two coordination regimes for ITAP, centralized and decentralized, and identify the structural assumptions that separate them.

3. In the centralized regime, we relate ITAP to scheduling and assignment problems with dependency constraints, and derive complexity-theoretic and approximation-theoretic consequences from this connection.

4. In the decentralized regime, we introduce the *Verifiable Allocation Market POSG* (VAMP), a partially observable stochastic game in which agents coordinate through market-mediated incentives over subtasks and intermediate results.

5. We show that equilibrium computation in VAMPs is intractable in general, which motivates the search for approximate solution methods rather than exact ones.

6. We develop a multiagent reinforcement learning framework for approximating useful low-regret behavior in VAMPs, combining sequence-based policies, centralized training with decentralized execution, and population-based training.

7. We evaluate the resulting methods on synthetic task-allocation environments under several market mechanisms, and we instantiate the framework in neural theorem proving within Lean.

Chapter 2

Centralized Multi-Agent Theorem Proving

With the motivation of modelling real-world mathematical development in mind, we first introduce a formalism of the core structure of mathematical tasks that will be used throughout this thesis. Inspired by proof-theoretic and type-theoretic semantics, we represent mathematical statements as formulas within a formal language, and the process of proving these statements as the resolution of proof obligations (sequents) that are defined by these formulas. We then introduce a library structure that models the evolving state of mathematical knowledge across multiple agents, each with private information.

Then with this formalism in place, we introduce the centralized multi-agent theorem proving problem, a POMDP in which a single centralized planner allocates compute across proof/-conjecture/publication actions for multiple agents with private information. We study this problem under the simplified environment of perfect public information, and study its complexity and approximation-theoretic properties by relating it to stochastic online scheduling problems with precedence constraints. We show that the problem is intractable in general, but identify special cases that are tractable and/or admit good approximations. We also discuss the limitations of this centralized formulation in the full environment of private information, and motivate the need for a decentralized formulation in which coordination is mediated by a market mechanism, which we study in the next chapter.

2.1 Background

2.1.1 Syntax

We consider the *formula* ϕ as the core syntactic primitive of (formal) mathematics. Namely, we represent mathematical theorem statements syntactically as a formula (i.e. object of type `Prop`) within some formal language Σ, such as Lean, Rocq, etc. We define $\mathcal{F}$ as the collection of all such formulas, and equip it with an involution $\neg : \mathcal{F} \to \mathcal{F}$; namely, $\neg(\neg\varphi) = \varphi$ for

all $\varphi \in \mathcal{F}$. Note that we will refer to formulas as "nodes" interchangably due to a natural graphical structure that will be discussed later.

2.1.2 Semantics

We now aim to represent the semantic meaning of a theorem statement within a particular context concisely. Towards this, we consider the *sequent*, given as an element of $\mathcal{S} := \mathcal{P}_{fin}(\mathcal{F}) \times \mathcal{F}$ and stylized as $[\Gamma \vdash \phi]$ (read: "Gamma *proves* phi"). Namely, such a sequent represents a *proof obligation*, i.e. the notion of proving a theorem statement ϕ (the *consequent*) within the *antecedent* context Γ. Such a proof obligation may be *resolved*, meaning proven.

2.1.3 Libraries

We assume the existence of a universal clock $t \in \mathbb{N}$ and set of agents $\mathcal{N}$, and use it to define a library $\mathcal{L}_\bullet = \{\mathcal{L}_t\}_{t\in\mathbb{N}}$ as a collection of discrete library states that evolves over time. At every discrete timestep $t \in \mathbb{N}$, each library state is given as a tuple

$$\mathcal{L}_t := (\mathcal{C}_t, \mathcal{RS}_t, \delta_t, \Theta_t) \in \mathscr{L}$$

where we partition $\mathcal{F} = \mathcal{C}_t \cup \mathcal{G}_t, \mathcal{C}_t \cap \mathcal{G}_t = \varnothing$ into *concrete* and *ghost* nodes. Intuitively, concrete nodes represent theorem statements that have been stated or conjectured, and ghost nodes represent all hypothetical, possible theorem statements that are yet to be conjectured or stated. $\mathcal{C}_t$ naturally induces a semantic analogue: let $\mathcal{CS}_t := \mathcal{P}_{fin}(\mathcal{C}_t) \times \mathcal{C}_t$ be the set of all *concrete sequents*, which contains a subset $\mathcal{RS}_t \subseteq \mathcal{CS}_t$ of *resolved sequents*, which represent the concrete nodes that have been proven (within a particular antecedent context, of course). Mathematically, a proof of a particular theorem in some context often explicitly depends on other lemmas within this context. In an effort to model this phenomenon, define $\delta_t : \mathcal{RS}_t \to \mathcal{P}_{fin}(\mathcal{F})$ as the dependency mapping. Note that in mathematics, there may exist many proofs of the same theorem with different dependencies for each, and one may find new proofs of the same theorem that are also valid. Thus, in the most general formulation, we only require that resolved sequents must remain resolved ($\mathcal{RS}_t \subseteq \mathcal{RS}_{t+1}$), though the underlying "proof" (moreso, the set of dependencies of this "proof") can change.
And lastly, we define $\Theta_t : \mathcal{RS}_t \to \mathbb{N}^+ \times \mathcal{N}$ to record the (discrete) solve-time associated to each resolved sequent, and the agent responsible for the resolution.
So, in other words, the library state $\mathcal{L}_t$ represents the collection of all stated theorem statements, the collection of proof witnesses within particular contexts for a subset of these statements, and the dependencies and ancillary metadata of these witnesses.

2.1.4 Library Invariants and Properties

Naturally, we impose a monotonicity invariant for libraries: one may not "unconjecture" or "unprove" a theorem. Formally, given a library $\mathcal{L}_\bullet = \{\mathcal{L}_t\}_{t\in\mathbb{N}} = \{(\mathcal{C}_t, \mathcal{RS}_t, \delta_t, \Theta_t)\}_{t\in\mathbb{N}}$, we require that for all $t \in \mathbb{N}$:

$$\mathcal{C}_t \subseteq \mathcal{C}_{t+1} \quad \text{and} \quad \mathcal{RS}_t \subseteq \mathcal{RS}_{t+1}$$

Additionally, we require that if one conjectures a theorem, they must also have automatically conjectured its converse; in other words:

$$\phi \in \mathcal{C}_t \iff \neg\phi \in \mathcal{C}_t$$

Similarly, one may not prove both a theorem and its converse true:

$$[\Gamma \vdash \phi] \in \mathcal{RS}_t \implies [\Gamma \vdash \neg\phi] \notin \mathcal{RS}_t$$

Semantically, we note reflexivity holds: $\phi \in \Gamma \implies [\Gamma \vdash \phi] \in \mathcal{RS}_t$. And for the sake of consistency, we require that for any $[\Gamma \vdash \phi] \in \mathcal{CS}_t$, $\psi \in \Gamma \implies \neg\psi \notin \Gamma$ (although we do not impose any full propositional semantics or richer logical theory behind these abstract formulas, preferring to leave them fully symbolic objects with various resolution and concreteness states, allowing inconsistent antecedents in our witnesses limits the scalability of our approach to deployment as a strategy module for multiagent theorem proving in a fully-defined formal language and type theory).

Note that this last condition implicitly redefines

$$\mathcal{CS}_t := \mathcal{P}_{fin}(\mathcal{C}_t)/\sim \times \mathcal{C}_t$$

where $\phi \sim \neg\phi$ is the equivalence relation generated by the negation operation. So, due to this consistency condition, we note that $\mathcal{RS}_t$ is upwards-closed, in that if $[\Gamma \vdash \phi] \in \mathcal{RS}_t$, then for all $\Gamma' \in \mathcal{P}_{fin}(\mathcal{C}_t)$, $\Gamma \subseteq \Gamma' \implies [\Gamma' \vdash \phi] \in \mathcal{RS}_t$.

Additionally, considering the dependency mapping, we note that intuitively δ_t must satisfy the property that $\delta_t([\Gamma \vdash \phi]) \subseteq \Gamma$. Moreover, one may use δ_t to induce a directed graph $D(\delta_t) := (\mathcal{CS}_t, \mathcal{E}_t)$, where

$$\mathcal{E}_t = \Big\{([\Gamma \vdash \phi], [\Gamma' \vdash \psi]) : \phi \in \delta_t([\Gamma' \vdash \psi]), \Gamma \subseteq \Gamma'\Big\}$$

We require that this induced directed graph $D(\delta_t)$ of δ_t must be acyclic.

2.1.5 Axioms and Fully Resolved Sequents

As an aside, we additionally note that although $[\Gamma \vdash \phi] \in \mathcal{RS}_t$ may be marked resolved, that does not mean that ϕ has a "error"-free proof within the current library state as its proof may depend on some still-unresolved lemmas. More concretely, we may define $\mathcal{FRS}_t \subseteq \mathcal{RS}_t$ as the collection of *fully resolved sequents*, which is defined inductively such that *axioms* $[\vdash \phi] \in \mathcal{FRS}_t$ and $[\Gamma \vdash \phi] \in \mathcal{FRS}_t$ if and only if for all $\phi' \in \delta_t([\Gamma \vdash \phi])$, there exists $\Gamma' \subseteq \Gamma$ such that $[\Gamma' \vdash \phi'] \in \mathcal{FRS}_t$.

2.1.6 Private and Shared Libraries

In the multiagent setting, each agent $\alpha \in \mathcal{N}$ maintains a *private library* $\mathcal{L}^\alpha_\bullet = \{\mathcal{L}^\alpha_t\}_{t\in\mathbb{N}}$ with

$$\mathcal{L}^\alpha_t := (\mathcal{C}^\alpha_t, \mathcal{RS}^\alpha_t, \delta^\alpha_t, \Theta^\alpha_t),$$

and the environment maintains a *shared (public) library* $\mathcal{L}^0_\bullet = \{\mathcal{L}^0_t\}_{t\in\mathbb{N}}$ with

$$\mathcal{L}^0_t := (\mathcal{C}^0_t, \mathcal{RS}^0_t, \delta^0_t, \Theta^0_t).$$

We additionally enforce the following inclusion invariant for all $t \in \mathbb{N}, \alpha \in \mathcal{N}$:

$$\mathcal{L}^0_t \subseteq \mathcal{L}^\alpha_t$$

meaning $\mathcal{C}^0_t \subseteq \mathcal{C}^\alpha_t$, $\mathcal{RS}^0_t \subseteq \mathcal{RS}^\alpha_t$, and for all $s \in \mathcal{RS}^0_t, \delta^0_t(s) = \delta^\alpha_t(s)$ and $\Theta^0_t(s) = \Theta^\alpha_t(s)$.

2.2 The Centralized MATP Problem

We now introduce the Centralized Multi-Agent Theorem Proving (MATP) problem, a POMDP in which a single centralized planner allocates compute across proof/conjecture/publication actions for multiple agents with private information.

2.2.1 Background: POMDP

We first recall the standard Markov Decision Process (MDP) formalism, which models a single agent sequential decision making problem with full state observability.

Definition 2.1 (Markov Decision Process)**.** A discrete-time Markov decision process is a tuple

$$\mathcal{M} = (\mathsf{S},\ \mathsf{A},\ T,\ T^0,\ c),$$

where S is a measurable state space, A is a measurable action space, $T(\cdot \mid s, a) \in \mathcal{P}(\mathsf{S})$ is a transition kernel, $T^0 \in \mathcal{P}(\mathsf{S})$ is an initial state distribution, and $c : \mathsf{S} \times \mathsf{A} \to \mathbb{R}$ is a

per-step cost function. At time $t = 0$, nature draws $s_0 \sim T^0$. At each subsequent time $t \geq 0$, the controller selects a_t, incurs cost $c(s_t, a_t)$, and the environment transitions to $s_{t+1} \sim T(\cdot \mid s_t, a_t)$.

One may generalize the MDP formalism to include partial observability, which is more appropriate for modelling the multiagent theorem proving problem in its full generality, as each agent has private information in the form of their private library state. The natural formulation of this problem is as a Partially Observable Markov Decision Process (POMDP), in which the agent does not directly observe the underlying state, but rather receives observations that are generated from the underlying state according to some observation function.

Definition 2.2 (POMDP)**.** A discrete-time partially observable Markov decision process is a tuple

$$\mathcal{P} = (\mathsf{S},\ \mathsf{A},\ \mathsf{O},\ T,\ T^0,\ Z,\ c),$$

for a state space S, a measurable space whose elements encode the full (hidden) state of the environment; an action space A, a measurable space of actions available to the controller; an observation space O, a measurable space of observations received by the controller; a transition kernel $T(\cdot \mid s, a) \in \mathcal{P}(\mathsf{S})$ specifying the distribution of the next state given a current state and action; an initial state distribution $T^0 \in \mathcal{P}(\mathsf{S})$; an observation kernel $Z(\cdot \mid s) \in \mathcal{P}(\mathsf{O})$ specifying the distribution of the observation received by the controller when the environment is in state s; and a per-step cost function $c : \mathsf{S} \times \mathsf{A} \to \mathbb{R}$.

The process evolves as follows. At time $t = 0$, nature draws a latent state $s_0 \sim T^0$ and the controller receives an observation $o_0 \sim Z(\cdot \mid s_0)$. At each subsequent time $t \geq 0$, the controller selects an action a_t, the environment transitions to a new state $s_{t+1} \sim T(\cdot \mid s_t, a_t)$, and the controller receives a new observation $o_{t+1} \sim Z(\cdot \mid s_{t+1})$.
The controller's observation history at time t is the tuple

$$h_t := (o_0,\ a_0,\ o_1,\ a_1,\ \ldots,\ a_{t-1},\ o_t),$$

which constitutes the controller's complete information record.

Definition 2.3 (Policy)**.** A randomized nonanticipative policy is a family of stochastic kernels $\pi = (\pi_t)_{t \geq 0}$ with $\pi_t : \mathsf{H}_t \to \mathcal{P}(\mathsf{A})$, where $\mathsf{H}_t := (\mathsf{O} \times \mathsf{A})^t \times \mathsf{O}$ is the space of observation histories of length t, such that $a_t \sim \pi_t(\cdot \mid h_t)$ depends only on the observation history up to time t. Equivalently, the action process $(a_t)_{t \geq 0}$ is adapted to the observation filtration $\mathcal{F}_t^{\mathrm{obs}} := \sigma(o_0, a_0, \ldots, a_{t-1}, o_t)$.

Remark (Belief state)*.* By standard POMDP theory, the posterior belief $b_t := \mathrm{Law}(s_t \mid h_t) \in \mathcal{P}(\mathsf{S})$ is a sufficient statistic of the observation history for optimal control: there exists an

optimal policy whose action at time t depends on h_t only through b_t. One may equivalently reformulate the POMDP as a fully observed MDP on the belief space $\mathcal{P}(\mathsf{S})$, the *belief MDP*, in which the state is b_t and the transition is given by Bayesian updating.

To solve a POMDP means to find a policy π that minimizes the expected total (discounted) cost:

$$J(\pi) := \mathbb{E}^{\pi}\left[\sum_{t=0}^{\infty} \gamma^t \, c(s_t, a_t) \;\middle|\; s_0 \sim T^0\right]$$

for a discount factor $\gamma \in [0, 1)$, or the expected total cost under a finite horizon H:

$$J_H(\pi) := \mathbb{E}^{\pi}\left[\sum_{t=0}^{H-1} c(s_t, a_t) \;\middle|\; s_0 \sim T^0\right].$$

2.2.2 Background: Action Primitives

We now specify the action primitives that the centralized planner may assign to individual agents. These primitives are standalone kernels and update maps that will be assembled into the POMDP transition structure.

2.2.2.1 Discrete kernels.

All stochastic primitives in this section have finite or countable output spaces, and may be treated as probability mass functions (pmfs). Concretely, for an input space X and discrete output space Y, a stochastic kernel is a map $K : Y \times X \to [0, 1]$ such that $\sum_{y \in Y} K(y \mid x) = 1$ for all $x \in X$.

2.2.2.2 Proof Kernel.

Fix an agent $\alpha \in \mathcal{N}$. The proof kernel models allocating a time budget to attempt to resolve a target sequent. Let the input space be

$$X_{\text{proof}} := \mathscr{L} \times \mathcal{S} \times \mathbb{N}^+,$$

where the components are a library state, a target sequent $s = [\Gamma \vdash \phi]$, and a discrete time budget τ. Let the output space be

$$Y_{\text{proof}} := \{(0, \bot)\} \;\cup\; \{(1, d) : d \in \mathcal{P}_{\text{fin}}(\mathcal{F})\},$$

where $(0, \bot)$ denotes failure, and $(1, d)$ denotes success together with a dependency set d. The proof kernel for agent α is a pmf

$$\mathcal{K}^{\alpha}_{\text{proof}} : Y_{\text{proof}} \times X_{\text{proof}} \to [0,1].$$

On admissible inputs $(\mathcal{L}, [\Gamma \vdash \phi], \tau)$, a successful output $(1, d)$ satisfies $d \subseteq \Gamma$.

2.2.2.3 Conjecture Kernel.

Fix an agent $\alpha \in \mathcal{N}$. The conjecture kernel models allocating a time budget to propose a new formula, anchored at an existing sequent. Let the input space be

$$X_{\text{conj}} := \mathscr{L} \times \mathcal{S} \times \mathbb{N}^{+},$$

and the output space be

$$Y_{\text{conj}} := \{(0, \perp)\} \ \cup \ \{(1, \psi) : \psi \in \mathcal{F}\}.$$

The conjecture kernel for agent α is a pmf $\mathcal{K}^{\alpha}_{\text{conj}} : Y_{\text{conj}} \times X_{\text{conj}} \to [0,1]$. Given admissible input $(\mathcal{L}, [\Gamma \vdash \phi], \tau)$ with $\mathcal{L} = (\mathcal{C}, \mathcal{RS}, \delta, \Theta)$ and a successful output $(1, \psi)$, we enforce $\psi \in \mathcal{F} \setminus \mathcal{C}$.

2.2.2.4 Dependency Closure for Publication.

Publication is a deterministic primitive that copies a closure-completed subset of an agent's private library into the shared library. Fix agent α and time t, and suppose δ^{α}_{t} induces the DAG $D(\delta^{\alpha}_{t}) = (\mathcal{CS}^{\alpha}_{t}, \mathcal{E}^{\alpha}_{t})$. For any set of target resolved sequents $R \subseteq \mathcal{RS}^{\alpha}_{t}$, define its *dependency closure*:

$$\begin{aligned} \text{cl}^{\alpha}_{t}(R) := \Big\{ & s \in \mathcal{RS}^{\alpha}_{t} : \exists\, r \in R \text{ such that there} \\ & \text{is a directed path } s \to r \text{ in } D(\delta^{\alpha}_{t}) \Big\} \end{aligned}$$

A publication payload is a pair $U^{\alpha}_{t} = (C, R)$ with $C \subseteq \mathcal{C}^{\alpha}_{t},\ R \subseteq \mathcal{RS}^{\alpha}_{t}$. Its closure-completion is $\overline{U^{\alpha}_{t}} := (\overline{C}, \overline{R})$ where $\overline{R} := \text{cl}^{\alpha}_{t}(R)$ and

$$\overline{C} := C \ \cup \bigcup_{[\Gamma \vdash \phi] \in \overline{R}} (\Gamma \cup \{\phi\}).$$

2.2.2.5 Query Model.

Each agent α maintains a stateful query model $Q^{\alpha}_{t} \in \mathcal{Q}$ that estimates prover performance. A query readout is a map $q^{\alpha} : \mathcal{Q} \times \mathcal{S} \to [0,1] \times \mathbb{N}$, returning $(\hat{p}^{\alpha}_{t}(s), \hat{\tau}^{\alpha}_{t}(s)) := q^{\alpha}(Q^{\alpha}_{t}, s)$, where $\hat{p}^{\alpha}_{t}(s)$ estimates the probability that sequent s is resolvable by agent α, and $\hat{\tau}^{\alpha}_{t}(s)$ estimates the expected time-to-solution.

The query model is updated via an operator $\Delta^\alpha : \mathcal{Q} \times \mathcal{D}(\mathcal{Q}) \to \mathcal{Q}$, where $\mathcal{D}(\mathcal{Q}) := \mathcal{S} \times (\mathbb{N}^+ \times \mathcal{N})$ is the space of query-training data. We enforce commutativity and associativity of Δ^α so that batch updates are well-defined and order-independent.

2.2.3 Definition

Fix a set of agents $\mathcal{N}$ and a target set of sequents $\Phi^* \subseteq \mathcal{S}$. The *Centralized Multi-Agent Theorem Proving* (MATP) problem is specified by the POMDP

$$\mathcal{P}_{\text{MATP}} = (\mathsf{S},\ \mathsf{A},\ \mathsf{O},\ T,\ T^0,\ Z,\ c)$$

defined as follows.

2.2.3.1 State Space S.

A state at time t is a tuple

$$s_t := \Big(\{\mathcal{L}_t^\alpha\}_{\alpha\in\mathcal{N}},\ \mathcal{L}_t^0,\ \{J_t^\alpha\}_{\alpha\in\mathcal{N}},\ \{Q_t^\alpha\}_{\alpha\in\mathcal{N}},\ \{\Omega_t^\alpha\}_{\alpha\in\mathcal{N}},\ \{U_t^\alpha\}_{\alpha\in\mathcal{N}}\Big) \in \mathsf{S},$$

where:

- $\mathcal{L}_t^\alpha \in \mathscr{L}$ is agent α's private library and $\mathcal{L}_t^0 \in \mathscr{L}$ is the shared public library, subject to the inclusion invariant $\mathcal{L}_t^0 \subseteq \mathcal{L}_t^\alpha$ for all $\alpha \in \mathcal{N}$.
- J_t^α is the *durative job descriptor*:

  $$J_t^\alpha \in \{\bot\} \ \cup\ \big(\{\text{Proof}, \text{Conj}\} \times \mathcal{S} \times \mathbb{N}^+ \times \mathbb{N}^+\big),$$

  with $\bot$ meaning idle and $(\cdot, s, \tau_{\text{rem}}, \tau_{\text{eff}})$ encoding a job of type Proof or Conj on target sequent s with τ_{rem} steps remaining in the current run and τ_{eff} the effective cumulative budget used to evaluate the kernel upon completion.
- $Q_t^\alpha \in \mathcal{Q}$ is agent α's query model state.
- $\Omega_t^\alpha \in \{\bot\} \ \sqcup\ (\mathcal{S} \times [0,1] \times \mathbb{N})$ is agent α's last query response, either empty $\bot$ or a triple $(s, \hat{p}, \hat{\tau})$.
- $U_t^\alpha : \mathcal{S} \times \{\text{proof}, \text{conj}\} \to \mathbb{N}$ records the total time already spent by agent α on proof or conjecture attempts for each sequent, with $U_0^\alpha \equiv 0$.

We take S to be the set of all such tuples satisfying the constituent objects' invariants.

2.2.3.2 Action Space A and Admissibility.

The centralized planner selects a *joint action* $a_t = (a_t^\alpha)_{\alpha\in\mathcal{N}} \in \mathsf{A}$, where $\mathsf{A} := \prod_{\alpha\in\mathcal{N}} \mathcal{A}^\alpha$ and each per-agent action space is

$$\begin{aligned}\mathcal{A}^\alpha := \{\text{Prove}\} &\times (\mathcal{S} \times \mathbb{N}^+) \\ \sqcup\ \{\text{Conj}\} &\times (\mathcal{S} \times \mathbb{N}^+) \\ \sqcup\ \{\text{Pub}\} &\times \big(\mathcal{P}_{\text{fin}}(\mathcal{F}) \times \mathcal{P}_{\text{fin}}(\mathcal{S})\big) \\ \sqcup\ \{\text{Qry}\} &\times \mathcal{S} \\ \sqcup\ \{\perp\}&.\end{aligned}$$

Define the admissibility predicate $\text{Adm} : \mathsf{S} \times \mathsf{A} \to \{0, 1\}$ componentwise: a joint action $a = (a^\alpha)_{\alpha\in\mathcal{N}}$ is admissible at state s if and only if $\text{Adm}^\alpha(s, a^\alpha) = 1$ for all $\alpha \in \mathcal{N}$, where:

- If $J_t^\alpha \neq \perp$, the agent is occupied and forced to no-op:

$$\text{Adm}^\alpha(s_t, a^\alpha) = \mathbf{1}[a^\alpha = \perp].$$

- If $J_t^\alpha = \perp$, the per-action admissibility conditions are:
  - $\text{Adm}^\alpha(s_t, \perp) = 1$ always.
  - For $a^\alpha = (\text{Prove}, (s, \tau))$ or $(\text{Conj}, (s, \tau))$: $\text{Adm}^\alpha(s_t, a^\alpha) = 1 \iff s \in \mathcal{CS}_t^\alpha$.
  - For $a^\alpha = (\text{Pub}, (C, R))$: $\text{Adm}^\alpha(s_t, a^\alpha) = 1 \iff C \subseteq \mathcal{C}_t^\alpha$ and $R \subseteq \mathcal{RS}_t^\alpha$.
  - For $a^\alpha = (\text{Qry}, s)$: $\text{Adm}^\alpha(s_t, a^\alpha) = 1 \iff s \in \mathcal{CS}_t^\alpha$.

Remark (Centralization and admissibility). A crucial subtlety: the admissibility predicate for agent α depends on α's private library $\mathcal{L}_t^\alpha$, which the centralized planner may not fully observe. Thus, the planner must reason about which actions are admissible for each agent based on its belief over the hidden state. This is a fundamental source of difficulty in the centralized formulation with private information, and is precisely what motivates the connection to mechanism design and the Nisan–Ronen conjecture in Section 2.4.5.

2.2.3.3 Observation Space O and Observation Kernel Z.

The centralized planner observes the public projection of the state:

$$o_t := \big(\mathcal{L}_t^0,\ \{\Omega_t^\alpha\}_{\alpha\in\mathcal{N}}\big) \in \mathsf{O}.$$

That is, the planner observes the shared library and the most recent query response from each agent. In particular, the private libraries $\{\mathcal{L}_t^\alpha \setminus \mathcal{L}_t^0\}_{\alpha\in\mathcal{N}}$, internal job states, and query model parameters are *hidden*.

The observation kernel is deterministic:

$$Z(o \mid s) = \mathbf{1}\big[\, o = \mathrm{Obs}(s) \,\big],$$

where $\mathrm{Obs} : \mathsf{S} \to \mathsf{O}$ computes the public projection as above.

Remark (Perfect public information). In the special case where all libraries are public—i.e., $\mathcal{L}_t^\alpha = \mathcal{L}_t^0$ for all $\alpha \in \mathcal{N}$ and all t—the observation o_t determines s_t and the POMDP reduces to a fully observed MDP. We refer to this as the *perfect public information* (PPI) setting and study it in Sections 2.3 and 2.4.

2.2.3.4 Transition Kernel T.

We define $T(\cdot \mid s_t, a_t)$ by a three-step generative construction: a deterministic pre-update, a stochastic job-completion draw, and a deterministic post-update.

Step 1: Deterministic Pre-Update. Given a state $s_t \in \mathsf{S}$ and admissible joint action $a_t \in \mathsf{A}$, define $\tilde{s}_t := \mathrm{Pre}(s_t, a_t)$ by applying the following edits in order:

1. **Decrement job timers.** For each $\alpha \in \mathcal{N}$: if $J_t^\alpha = \bot$, leave it unchanged; if $J_t^\alpha = (\cdot, s, \tau_{\mathrm{rem}}, \tau_{\mathrm{eff}})$ with $\tau_{\mathrm{rem}} > 0$, set $\tau_{\mathrm{rem}} \leftarrow \tau_{\mathrm{rem}} - 1$.

2. **Start new jobs.** For each idle agent α (i.e., $J_t^\alpha = \bot$):

   - If $a_t^\alpha = (\mathrm{Prove}, (s, \tau))$, set $J^\alpha \leftarrow (\mathrm{Proof}, s, \tau, \tau + U_t^\alpha(s, \mathrm{proof}))$.
   - If $a_t^\alpha = (\mathrm{Conj}, (s, \tau))$, set $J^\alpha \leftarrow (\mathrm{Conj}, s, \tau, \tau + U_t^\alpha(s, \mathrm{conj}))$.

3. **Apply publications.** For each $\alpha \in \mathcal{N}$ with $a_t^\alpha = (\mathrm{Pub}, (C^\alpha, R^\alpha))$: compute the closure-completion $\overline{(C^\alpha, R^\alpha)} = (\overline{C}^\alpha, \overline{R}^\alpha)$ and update:

   $$\mathcal{C}^0 \leftarrow \mathcal{C}^0 \cup \textstyle\bigcup_\alpha \overline{C}^\alpha, \qquad \mathcal{RS}^0 \leftarrow \mathcal{RS}^0 \cup \textstyle\bigcup_\alpha \overline{R}^\alpha.$$

   For each newly resolved $s \in \mathcal{RS}^0_{\mathrm{new}} := \mathcal{RS}^0 \setminus \mathcal{RS}^0_{\mathrm{old}}$, set $\delta^0(s) \leftarrow \delta^\alpha(s)$ and $\Theta^0(s) \leftarrow \Theta^\alpha(s)$. Propagate to each agent β:

   $$\mathcal{C}^\beta \leftarrow \mathcal{C}^\beta \cup (\mathcal{C}^0 \setminus \mathcal{C}^0_{\mathrm{old}}), \quad \mathcal{RS}^\beta \leftarrow \mathcal{RS}^\beta \cup \mathcal{RS}^0_{\mathrm{new}},$$

   and for each $s \in \mathcal{RS}^0_{\mathrm{new}}$: $\delta^\beta(s) \leftarrow \delta^0(s)$, $\Theta^\beta(s) \leftarrow \Theta^0(s)$, and $Q^\beta \leftarrow \Delta^\beta(Q^\beta, (s, \Theta^0(s)))$.

4. **Update query buffers.** For each $\alpha \in \mathcal{N}$: if $a_t^\alpha = (\mathrm{Qry}, s)$, set $\Omega^\alpha \leftarrow (s, \hat{p}, \max\{0, \hat{\tau} - U_t^\alpha(s, \mathrm{proof})\})$ where $(\hat{p}, \hat{\tau}) := q^\alpha(Q^\alpha, s)$; otherwise set $\Omega^\alpha \leftarrow \bot$.

Step 2: Stochastic Job Completion. Given $\tilde{s}_t$, define the sets of completing agents:

$$\mathsf{Comp}_{\text{proof}}(\tilde{s}_t) = \{\alpha : J^\alpha(\tilde{s}_t) = (\text{Proof}, s, 0, \tau) \text{ for some } s, \tau\},$$

$$\mathsf{Comp}_{\text{conj}}(\tilde{s}_t) = \{\alpha : J^\alpha(\tilde{s}_t) = (\text{Conj}, s, 0, \tau) \text{ for some } s, \tau\}.$$

For each completing agent, we draw an independent outcome from the appropriate kernel. The joint outcome space is

$$Y(\tilde{s}_t) := \prod_{\alpha \in \mathsf{Comp}_{\text{proof}}(\tilde{s}_t)} Y_{\text{proof}} \times \prod_{\alpha \in \mathsf{Comp}_{\text{conj}}(\tilde{s}_t)} Y_{\text{conj}},$$

equipped with the product measure

$$\begin{aligned}\mathbb{K}_{\tilde{s}_t}(dy) := &\prod_{\alpha \in \mathsf{Comp}_{\text{proof}}(\tilde{s}_t)} \mathcal{K}^\alpha_{\text{proof}}\big(dy^\alpha \mid \mathcal{L}^\alpha(\tilde{s}_t), s^\alpha_{\text{proof}}, \tau^\alpha_{\text{proof}}\big) \\ &\cdot \prod_{\alpha \in \mathsf{Comp}_{\text{conj}}(\tilde{s}_t)} \mathcal{K}^\alpha_{\text{conj}}\big(dy^\alpha \mid \mathcal{L}^\alpha(\tilde{s}_t), s^\alpha_{\text{conj}}, \tau^\alpha_{\text{conj}}\big),\end{aligned}$$

where $(s^\alpha_\cdot, \tau^\alpha_\cdot)$ are read from $J^\alpha(\tilde{s}_t)$.

Step 3: Deterministic Post-Update. Given $\tilde{s}_t$ and realized outcomes $y \in Y(\tilde{s}_t)$, define $s_{t+1} := \text{Post}(\tilde{s}_t, y)$ by applying:

1. **Resolve proof completions.** For each $\alpha \in \mathsf{Comp}_{\text{proof}}(\tilde{s}_t)$ with outcome $y^\alpha = (b^\alpha, z^\alpha)$ and stored target/budget (s^α, τ^α): set $U^\alpha(s^\alpha, \text{proof}) \leftarrow \tau^\alpha$. If $b^\alpha = 1$ and $z^\alpha = d$:
   - $\mathcal{RS}^\alpha \leftarrow \mathcal{RS}^\alpha \cup \{s^\alpha\}$,
   - $\delta^\alpha(s^\alpha) \leftarrow d$ (subject to $d \subseteq \Gamma$ and acyclicity),
   - $\Theta^\alpha(s^\alpha) \leftarrow (\tau^\alpha, \alpha)$,
   - $Q^\alpha \leftarrow \Delta^\alpha(Q^\alpha, (s^\alpha, \Theta^\alpha(s^\alpha)))$.
2. **Resolve conjecture completions.** For each $\alpha \in \mathsf{Comp}_{\text{conj}}(\tilde{s}_t)$ with outcome $y^\alpha = (b^\alpha, z^\alpha)$: set $U^\alpha(s^\alpha, \text{conj}) \leftarrow \tau^\alpha$. If $b^\alpha = 1$ and $z^\alpha = \psi$: $\mathcal{C}^\alpha \leftarrow \mathcal{C}^\alpha \cup \{\psi, \neg\psi\}$.
3. **Clear completed jobs.** For each $\alpha \in \mathsf{Comp}_{\text{proof}}(\tilde{s}_t) \cup \mathsf{Comp}_{\text{conj}}(\tilde{s}_t)$: set $J^\alpha \leftarrow \perp$.

2.2.3.5 Full transition kernel.

The transition kernel is the pushforward:

$$T(U \mid s_t, a_t) \;=\; \int_{Y(\tilde{s}_t)} \mathbf{1}\big[\mathrm{Post}(\tilde{s}_t, y) \in U\big]\, \mathbb{K}_{\tilde{s}_t}(dy),$$

for measurable $U \subseteq \mathsf{S}$, where $\tilde{s}_t := \mathrm{Pre}(s_t, a_t)$.

2.2.3.6 Cost Function.

Fix a target set $\Phi^* \subseteq \mathcal{CS}_0^0$ of sequents that the planner aims to fully resolve in the shared library. We define the per-step cost as

$$c(s_t, a_t) := \sum_{s \in \Phi^*} \mathbf{1}\big[s \notin \mathcal{FRS}_t^0\big],$$

the number of target sequents not yet fully resolved in the shared library at time t.

Remark (Cost variants). Under this cost function, the finite-horizon objective $J_H(\pi) = \sum_{t=0}^{H-1} \mathbb{E}[c(s_t, a_t)]$ is minimized by policies that resolve target sequents as early as possible, yielding a natural connection to weighted completion time in scheduling theory. For a pure *makespan* objective (minimizing the time at which all targets are resolved), one may instead consider the stopping-time formulation

$$\mathrm{makespan}(\pi) := \inf\big\{t \geq 0 : \Phi^* \subseteq \mathcal{FRS}_t^0\big\}$$

and seek to minimize $\mathbb{E}^\pi[\mathrm{makespan}(\pi)]$. The discounted formulation with c as above interpolates between these via the discount factor γ.

2.2.3.7 Initial distribution T^0.

T^0 is a distribution over initial states s_0 specifying initial private and shared libraries (satisfying $\mathcal{L}_0^0 \subseteq \mathcal{L}_0^\alpha$), idle job statuses ($J_0^\alpha = \perp$ for all α), zero cumulative-effort maps ($U_0^\alpha \equiv 0$), initial query models $Q_0^\alpha \in \mathcal{Q}$, and empty query buffers ($\Omega_0^\alpha = \perp$), supported on states satisfying all invariants.

2.2.4 Example

TODO REPLACE WITH A TRIANGLE THAT SHOWS PARALLEL WORK WITH STOCHASTICITY

We work through a simple instance to illustrate the centralized MATP formulation.

2.2.4.1 Setup.

Let $\mathcal{N} = \{\alpha_1, \alpha_2\}$ be two agents with identical, deterministic proof kernels. The shared library $\mathcal{L}_0^0$ contains three concrete formulas $\{A, B, C\}$ (and their negations), with the single

axiom $[\vdash A] \in \mathcal{FRS}_0^0$. The planner's target is $\Phi^* = \{[\vdash C]\}$. The dependency structure of C requires two intermediate lemmas:

$$[A \vdash B] \quad \text{and} \quad [A, B \vdash C],$$

meaning that proving C from axioms requires first proving B from A, then C from A and B. All libraries are public (the PPI setting of Remark 2.2.3.3), so the POMDP reduces to an MDP. Each proof attempt takes exactly $\tau = 1$ step and succeeds deterministically.

2.2.4.2 Optimal plan.

At $t = 0$, the planner assigns $\alpha_1 \mapsto (\text{Prove}, ([A \vdash B], 1))$ and $\alpha_2 \mapsto \perp$. Agent α_2 is forced to idle because it cannot yet attempt $[A, B \vdash C]$, as B has not been proven. At $t = 1$, α_1's proof completes, resolving $[A \vdash B]$ in the (fully public) library. The planner then publishes it and assigns either agent to $(\text{Prove}, ([A, B \vdash C], 1))$, which resolves C at $t = 2$.

2.2.4.3 Discussion.

The sequential dependency $A \to B \to C$ prevents any parallel speedup despite having two agents, illustrating how the precedence structure of the dependency DAG constrains the planner's scheduling freedom. This observation motivates the reduction to stochastic scheduling with precedence constraints developed in Section 2.3.

Even in this small instance, if the libraries were private (i.e., α_1 discovers $[A \vdash B]$ but has not yet published it), the planner would need to decide *when* to instruct α_1 to publish, trading off the publication cost (one timestep) against enabling α_2. The planner's inability to directly observe α_1's private library in the general (non-PPI) setting further complicates scheduling, as the planner must infer what each agent has privately proven from the history of query responses and publications.

2.3 MATP and Scheduling Theory

We now analyze the centralized MATP problem under two simplifying restrictions that expose its combinatorial core. First, we assume the *perfect public information* (PPI) setting of Remark 2.2.3.3, in which all libraries are public ($\mathcal{L}_t^\alpha = \mathcal{L}_t^0$ for all α, t); this eliminates partial observability and reduces the POMDP to an MDP. Second, we disable conjecturing, so that the set of concrete formulas is fixed. Under these two restrictions, the centralized MATP acquires a natural interpretation as a stochastic scheduling problem with latent dependencies, allowing us to import complexity and approximation results from scheduling theory, while

also identifying MATP-specific structural features that go beyond the classical scheduling setting.

2.3.1 Non-Conjecturing PPI MATP

Definition 2.4 (Non-Conjecturing PPI MATP)**.** A *Non-Conjecturing PPI MATP* instance is a centralized MATP instance $\mathcal{P}_{\text{MATP}}$ subject to:

1. **Perfect public information (PPI).** $\mathcal{L}_t^\alpha = \mathcal{L}_t^0$ for all $\alpha \in \mathcal{N}$, $t \in \mathbb{N}$. Publication is vacuous, and the POMDP reduces to a fully observed MDP.

2. **No conjecturing.** The conjecture action is disabled, and $\mathcal{C}_0 = \mathcal{F}$: every formula is already concrete at initialization. Thus $\mathcal{CS}_0 = \mathcal{P}_{\text{fin}}(\mathcal{F})/\sim \ \times \ \mathcal{F}$ is the full set of concrete sequents, fixed for all time.

3. **Unique dependencies.** For each sequent $s \in \mathcal{S}$, the dependency set $d(s) \subseteq \mathcal{F}$ produced upon successful proof is unique: there is a single deterministic function $\delta^* : \mathcal{S} \to \mathcal{P}_{\text{fin}}(\mathcal{F})$ such that whenever any agent α successfully resolves s (i.e., $(1, d) \sim \mathcal{K}^\alpha_{\text{proof}}(\cdot \mid \mathcal{L}, s, \tau)$), we have $d = \delta^*(s)$. The function δ^* is *latent*: the planner does not know $\delta^*(s)$ until s is resolved.

2.3.1.1 The latent task set.

Since dependencies are unique and deterministic, the set of sequents that must ultimately be resolved to fully resolve the target set Φ^* is itself deterministic—though unknown to the planner.

Definition 2.5 (Dependency closure and latent task set)**.** For a target set $\Phi^* \subseteq \mathcal{CS}_0$, define the *latent task set* $\mathcal{T}^* = \text{cl}(\Phi^*)$ as the smallest set satisfying:

1. $\Phi^* \subseteq \mathcal{T}^*$.

2. For every $s = [\Gamma \vdash \phi] \in \mathcal{T}^*$ that is not already an axiom, and every $\psi \in \delta^*(s)$, there exists a sequent $s' = [\Gamma' \vdash \psi]$ with $\Gamma' \subseteq \Gamma$ such that $s' \in \mathcal{T}^*$.

By the acyclicity invariant on δ^*, this closure is well-defined and finite (assuming Φ^* is finite and δ^* has finite image).

We emphasize that $\mathcal{T}^*$ is a deterministic set, fixed by Φ^* and the latent function δ^*. However, the planner does not know $\mathcal{T}^*$ at time 0. Instead, $\mathcal{T}^*$ is revealed incrementally: when the planner resolves some $s \in \mathcal{T}^*$, the dependency set $\delta^*(s)$ becomes visible, and the planner learns which additional sequents must be resolved.

Remark (No operational precedence). A key structural feature: the planner may attempt to resolve *any* concrete sequent at any time. There is no operational precedence constraint requiring dependencies to be resolved before the sequent itself. The constraint is purely on the *objective*: a target sequent $s \in \Phi^*$ is *fully resolved* (i.e., $s \in \mathcal{FRS}_t$) only when s is resolved *and* all sequents in its (transitive) dependency closure are fully resolved. Thus the dependency structure constrains the *completion criterion*, not the *execution order*.

This is fundamentally different from classical scheduling with precedence constraints, where predecessor jobs must complete before successor jobs may begin. **TODO FIX No? i mean technically one can attempt to prove whatever they like whenever they like, things won't be fully resolved until the dependencies are resolved. And you find what the dependencies are as you prove things, so you can attempt to prove things that turn out to be irrelevant, but you won't get credit for them until the dependencies are resolved. so idk**

2.3.1.2 Information structure and revelation.

The planner's information at time t consists of the full public library state $\mathcal{L}_t^0$, including all resolved sequents $\mathcal{RS}_t^0$ and their (now-revealed) dependency sets $\{\delta^*(s) : s \in \mathcal{RS}_t^0\}$. The latent information is: (i) the dependency sets $\delta^*(s)$ for unresolved sequents $s \notin \mathcal{RS}_t^0$, and (ii) the realized processing times for incomplete proof attempts.

The query model provides noisy ex-ante estimates: for any concrete sequent s, the planner may query agent α to obtain $(\hat{p}, \hat{\tau}) = q^\alpha(Q_t^\alpha, s)$, estimating the probability that s is resolvable by α and the expected time to do so. These estimates are updated as proofs complete and the query model state Q_t^α is refined. This allows the planner to form beliefs about which sequents are likely to be in $\mathcal{T}^*$ and how difficult they are, enabling *speculative* proof attempts on sequents that may turn out to be needed.

2.3.2 Scheduling Interpretation

The Non-Conjecturing PPI MATP admits a natural interpretation as a stochastic scheduling problem, but with important structural differences from the classical setting.

2.3.2.1 Correspondence.

Fix a Non-Conjecturing PPI MATP instance with $m = |\mathcal{N}|$ agents and latent task set $\mathcal{T}^*$. The scheduling interpretation is:

MATP	Scheduling
Agent $\alpha \in \mathcal{N}$	Machine M_α (unrelated)
Sequent $s \in \mathcal{T}^*$	Job J_s
Time-to-solution T_s^α	Processing time $P_{\alpha s}$
Target set Φ^*	Target job set $\mathcal{J}^{\text{target}}$
Dependency $\psi \in \delta^*(s)$	Spawned job upon completion of J_s
Cumulative effort $U_t^\alpha(s)$	Preemptive progress (machine-specific)
OR-redundancy	Multiple machines on same job

2.3.2.2 Stochastic processing times.

For each job J_s and machine M_α, the processing time is the random variable T_s^α induced by the proof kernel:

$$\Pr(T_s^\alpha \leq \tau) = p_\tau^\alpha(s) := \sum_{d \in \mathcal{P}_{\text{fin}}(\mathcal{F})} \mathcal{K}_{\text{proof}}^\alpha\big((1,d) \mid \mathcal{L}_0^0, s, \tau\big).$$

If $p_\infty^\alpha(s) < 1$, the processing time is infinite with positive probability, corresponding to agent α being unable to resolve s.

2.3.2.3 Key departures from classical scheduling.

While the correspondence above is suggestive, the Non-Conjecturing PPI MATP differs from classical scheduling in three essential ways:

1. **No operational precedence, latent dependency revelation.** The job set $\mathcal{T}^*$ is not known to the scheduler at time 0. Only the target jobs Φ^* are known initially. When job J_s completes, the dependency set $\delta^*(s)$ is revealed, and the corresponding jobs $\{J_{s'} : s' \text{ induced by } \delta^*(s)\}$ enter the scheduler's known task queue. This is a *job-spawning* model: completing a job may generate new jobs. There is no restriction on the order in which jobs may be *started*—the scheduler may attempt any concrete sequent at any time.

2. **OR-redundancy.** Multiple agents may simultaneously attempt the same sequent. The sequent is resolved when *any* agent succeeds, and the effective processing time is $T_s^{\text{OR}} = \min_{\alpha \in A_s} T_s^\alpha$ for the set A_s of agents assigned to s. This is absent in standard scheduling, where each job is processed by exactly one machine per timestep.

3. **Machine-specific, non-transferable progress.** When agent α spends time τ on sequent s without success, the cumulative effort $U_t^\alpha(s, \text{proof})$ is updated and the next attempt by α on s benefits from the accumulated budget. However, agent β's progress on s is independent: U^β does not benefit from α's efforts. Progress is preemptible (an agent may pause and resume) but not transferable across agents. **TODO FIX: Maybe want to change def to allow for inter-machine progress transfer? Or maybe not bc private info is hard to transfer?**

2.3.2.4 Revelation DAG.

Define the *revelation DAG* $G^{\text{rev}} = (\mathcal{T}^*, E^{\text{rev}})$ by

$$E^{\text{rev}} = \{(s, s') : s' \text{ is induced by some } \psi \in \delta^*(s)\},$$

i.e., there is an edge from s to each job spawned by s's completion. A *revelation chain* is a directed path $s_0 \to s_1 \to \cdots \to s_k$ in G^{rev}, with the interpretation that s_{i+1} becomes known to the planner only after s_i is resolved. The acyclicity of δ^* ensures G^{rev} is a DAG. Denote its depth by $\text{depth}(G^{\text{rev}})$.

2.4 Complexity and Approximability

We now establish the computational complexity of the Non-Conjecturing PPI MATP and develop approximation bounds. Throughout, $m = |\mathcal{N}|$ is the number of agents, $n^* = |\mathcal{T}^*|$ is the size of the latent task set, and $n_0 = |\Phi^*|$ is the number of initial targets.

2.4.1 Complexity Results

2.4.1.1 Embedding of classical scheduling.

The Non-Conjecturing PPI MATP contains several classical scheduling problems as special cases, obtained by restricting the dependency structure and agent heterogeneity.

Proposition 2.1 (Scheduling embeddings)**.** *The following scheduling problems embed into the Non-Conjecturing PPI MATP:*

1. $P \| C_{\max}$ ***(identical parallel machines, makespan).*** *Take all agents identical ($\mathcal{K}_{\text{proof}}^{\alpha_i} = \mathcal{K}_{\text{proof}}^{\alpha_j}$), deterministic unit-success proof kernels, $\Phi^* = \mathcal{T}^*$ (no latent tasks), and $\delta^*(s) = \varnothing$ for all s (trivial dependencies). Then the MATP reduces to assigning n^* jobs with known deterministic processing times to m identical machines to minimize makespan.*

2. $R\|\,C_{\max}$ ***(unrelated machines, makespan).*** *As above but with heterogeneous proof kernels.*

3. $P \mid \text{prec} \mid C_{\max}$ ***(identical machines, precedence, makespan).*** *Take identical agents, $\delta^*(s) = \varnothing$ for all s, but introduce operational precedence by encoding it as follows: include "sentinel" sequents whose role is to enforce ordering. Specifically, if job J_j must precede J_k, include a sentinel sequent s_{jk} with $\delta^*(s_{jk}) = \{\phi_j\}$ (where ϕ_j is the consequent of J_j) and make s_{jk} a dependency of a target that also depends on J_k's consequent. The unique-dependency and latent-revelation structure then enforces the desired precedence: s_{jk} cannot be marked fully resolved until J_j completes, which in turn blocks full resolution of any target depending on s_{jk}.*

   In fact, this encoding is unnecessary in the special case where $\delta^*(s) = \varnothing$ for all s and all targets are independent, but becomes relevant when one considers the full dependency structure.

The embedding of $P\|\,C_{\max}$ is the most direct and yields the following.

Theorem 2.2 (NP-hardness of MATP)**.** *The decision problem for the Non-Conjecturing PPI MATP—"given an instance $\mathcal{P}$ and threshold T, is $\inf_\pi \mathbb{E}^\pi[C_{\max}] \le T$?"—is NP-hard, even restricted to:*

- *identical agents,*
- *deterministic proof kernels (processing times are deterministic),*
- *trivial dependencies ($\delta^*(s) = \varnothing$ for all s), and*
- *single-assignment (no OR-redundancy).*

More precisely, $P\|\,C_{\max}$ reduces in polynomial time to this restricted MATP. Since $P\|\,C_{\max}$ is strongly NP-hard for variable m (and weakly NP-hard for $m = 2$), the same holds for MATP.

Proof. Given an instance of $P\|\,C_{\max}$ with n jobs of processing times $p_1, \ldots, p_n$ on m identical machines, construct a Non-Conjecturing PPI MATP instance with m identical agents, target set $\Phi^* = \{s_1, \ldots, s_n\}$, and deterministic proof kernels such that agent α resolves s_j in exactly p_j time steps (i.e., $\mathcal{K}^\alpha_{\text{proof}}((1, \varnothing) \mid \mathcal{L}, s_j, \tau) = \mathbf{1}[\tau \ge p_j]$). Set $\delta^*(s_j) = \varnothing$ for all j. Then every resolved sequent is automatically fully resolved, and the MATP makespan equals the scheduling makespan. The reduction is polynomial in the input size. □

2.4.1.2 State space and MDP complexity.

Under PPI, the MATP is a fully observed MDP, and in principle one may apply standard MDP algorithms (value iteration, policy iteration) to compute the optimal policy. However, the state space is exponentially large in the problem parameters.

Proposition 2.3 (State space exponentiality)**.** *The state space* S *of the Non-Conjecturing PPI MATP MDP has cardinality at least* 2^{n^*}*, where* $n^* = |\mathcal{T}^*|$ *is the size of the latent task set. Specifically, the component* $\mathcal{RS}_t^0 \subseteq \mathcal{CS}_0$ *ranges over* 2^{n^*} *possible resolved-sequent sets (restricted to* $\mathcal{T}^*$*), and these are all reachable.*

Proof. Since any subset of $\mathcal{T}^*$ can arise as the resolved set at some time t (by choosing which sequents to prove and in what order), the state space has at least 2^{n^*} states. The additional job-descriptor and cumulative-effort components further enlarge the state space. □

Corollary 2.3.1 (Exact solution complexity)**.** *Solving the Non-Conjecturing PPI MATP MDP exactly (via value iteration or linear programming over the state space) requires time* $\mathrm{poly}(|\mathsf{S}|, |\mathsf{A}|) = 2^{\Omega(n^*)}$*, and is thus intractable for large task sets unless the MDP has exploitable structure (e.g., factored representations).*

Remark (PSPACE membership for the decision problem)*.* The decision problem "$\inf_\pi \mathbb{E}^\pi[C_{\max}] \leq T$?" for the stochastic MATP MDP lies in PSPACE: one can enumerate policies and simulate trajectories in polynomial space, maintaining only the current state and running cost. Combined with Theorem 2.2, the decision problem is NP-hard and in PSPACE. We conjecture it is PSPACE-complete, as the stochastic processing times and latent dependency revelation introduce planning-under-uncertainty structure analogous to stochastic reachability problems, but we do not prove this here.

2.4.2 Lower Bounds

We now establish information-theoretic lower bounds on the optimal makespan. These bounds are *universal*: they apply to every policy, including policies that speculatively prove sequents before their necessity is confirmed.

Fix a Non-Conjecturing PPI MATP instance with m agents, target set Φ^*, and latent task set $\mathcal{T}^*$.

Definition 2.6 (Work and serial depth)**.** For a realization ω (which determines all processing times), define:

- The *work* is the total processing time across all tasks in the latent task set, under an optimal single-assignment:

$$W(\omega) := \sum_{s \in \mathcal{T}^*} \min_{\alpha \in \mathcal{N}} T_s^\alpha(\omega).$$

- The *serial depth* is the maximum total processing time along any revelation chain:

$$D(\omega) := \max_{\substack{s_0 \to s_1 \to \cdots \to s_k \\ \text{in } G^{\text{rev}}}} \sum_{i=0}^{k} \min_{\alpha \in \mathcal{N}} T_{s_i}^\alpha(\omega).$$

Theorem 2.4 (Work and serial depth lower bounds)**.** *For any Non-Conjecturing PPI MATP instance and any admissible policy π:*

$$\mathbb{E}^\pi[C_{\max}] \;\geq\; \max\left\{\frac{\mathbb{E}[W]}{m},\;\; \mathbb{E}[D]\right\}.$$

Proof. **Work bound.** For each realization ω, every task $s \in \mathcal{T}^*$ must be resolved by some agent. Even if each task is resolved by the fastest agent (giving processing time $\min_\alpha T_s^\alpha(\omega)$), the total work is $W(\omega)$. With m agents working in parallel, the makespan is at least $W(\omega)/m$. Taking expectations: $\mathbb{E}^\pi[C_{\max}] \geq \mathbb{E}[W]/m$.

Serial depth bound. Consider any revelation chain $s_0 \to s_1 \to \cdots \to s_k$ in G^{rev}. By definition, s_{i+1} is a dependency of s_i, and $\delta^*(s_i)$ is revealed only when s_i is resolved. Thus s_{i+1} is not known to be required until time C_{s_i} (the completion time of s_i). Therefore, s_{i+1} cannot be completed before time $C_{s_i} + \min_\alpha T_{s_{i+1}}^\alpha(\omega)$ at the earliest (assigning the fastest available agent immediately upon revelation). Iterating: the completion of the chain requires time at least $\sum_{i=0}^{k} \min_\alpha T_{s_i}^\alpha(\omega)$. Since all tasks in the chain must be resolved for s_0 to be fully resolved (and $s_0 \in \Phi^*$ implies all its descendants are in $\mathcal{T}^*$), the makespan is at least $D(\omega)$. Taking expectations gives the bound. □

Remark (Tightness of the serial depth bound)*.* The serial depth bound is tight in the following sense: if the revelation DAG has a single chain $s_0 \to s_1 \to \cdots \to s_k$ with all other tasks having trivial dependencies, then no policy (not even one with perfect foresight about δ^*) can fully resolve s_0 faster than $\sum_i \min_\alpha T_{s_i}^\alpha(\omega)$, because the existence of s_{i+1} is not known until s_i completes.

Remark (Speculation and the serial depth bound)*.* One might wonder whether a policy that *speculatively* proves sequents—resolving s_{i+1} *before* s_i completes, on the guess that s_{i+1} will be needed—could beat the serial depth bound. It cannot: by assumption, δ^* is latent, and the planner does not know $s_{i+1} \in \delta^*(s_i)$ until s_i is resolved. If the planner speculatively proves some sequent s' that turns out not to be in $\mathcal{T}^*$, this work is wasted. If s' does turn out to be needed, the speculative work is useful, but the planner still cannot resolve s' before it is discovered to be in $\mathcal{T}^*$—the resolution may have already occurred speculatively, but

the *revelation* of s' as a required task still occurs only upon completion of its parent in the revelation DAG.

More precisely: speculation can help the *work bound* (by resolving tasks that will later be needed, before they are officially revealed). But speculation cannot help the *serial depth bound*, because the serial depth counts the sum of processing times along a chain where each link is only discovered upon completion of the previous link—and if a speculative proof of s_{i+1} was already completed before s_i, then $\min_\alpha T^\alpha_{s_{i+1}} = 0$ in the chain, and the serial depth adjusts accordingly.

To formalize: define the *residual serial depth* under policy π as the serial depth computed with processing times replaced by the *residual* time $\max\{0, \min_\alpha T^\alpha_{s_i}(\omega) - U^\alpha_{C_{s_{i-1}}}(s_i)\}$, accounting for any speculative work already completed. The bound $C_{\max}(\omega) \geq D_{\text{res}}(\omega)$ still holds, and in the worst case (no useful speculation), $D_{\text{res}} = D$.

2.4.3 Approximation Algorithms

We now develop approximation guarantees for the Non-Conjecturing PPI MATP. The natural algorithmic paradigm is *reactive scheduling*: maintain a queue of known required tasks (initialized to Φ^*, expanded as dependencies are revealed), and apply a scheduling heuristic to the current queue.

Definition 2.7 (Reactive list scheduling)**.** Fix a priority function $\sigma : \mathcal{CS}_0 \to \mathbb{R}$ (lower values indicate higher priority). The *Reactive List Scheduling* (RLS) policy operates as follows:

1. Initialize the *known task queue* $\mathcal{K}_0 := \Phi^*$.
2. At each timestep t: for each idle agent α, assign it to the highest-priority task in $\mathcal{K}_t$ that is not yet resolved and not currently being worked on by another agent (under single-assignment), or $\perp$ if no such task exists.
3. When a task s is resolved at time t, observe $\delta^*(s)$ and update:

$$\mathcal{K}_{t+1} \leftarrow \mathcal{K}_t \ \cup \ \{s' : s' \text{ is induced by } \psi \in \delta^*(s),\ s' \notin \mathcal{RS}^0_t\}.$$

Remark (OR-redundant variant)*.* In the OR-redundant variant of RLS, step (2) allows multiple idle agents to be assigned to the same task. This is advantageous when $|\mathcal{K}_t| < m$ (fewer known tasks than agents), as otherwise agents would idle.

2.4.3.1 Identical agents, single-assignment.

We first analyze the simplest nontrivial case: m identical agents with single-assignment.

Theorem 2.5 (RLS approximation for identical agents)**.** *Consider a Non-Conjecturing PPI MATP instance with m identical agents (all proof kernels equal) and single-assignment. Let* RLS *denote the reactive list scheduling policy with an arbitrary priority function. Then for each realization ω:*

$$C_{\max}^{\mathrm{RLS}}(\omega) \;\le\; \frac{W(\omega)}{m} + \left(1 - \frac{1}{m}\right) D(\omega),$$

where $W(\omega) = \sum_{s \in \mathcal{T}^*} T_s(\omega)$ and $D(\omega)$ is the serial depth. Consequently:

$$\mathbb{E}[C_{\max}^{\mathrm{RLS}}] \;\le\; \left(2 - \frac{1}{m}\right) \mathbb{E}[\mathrm{OPT}].$$

Proof. Fix a realization ω and let s^* be the last task to complete under RLS. Since all tasks in $\mathcal{T}^*$ must be resolved, and s^* is the last to finish, we have $C_{\max}^{\mathrm{RLS}}(\omega) = C_{s^*}(\omega)$.
Partition the interval $[0, C_{s^*}]$ into:

- **Busy intervals**: timesteps at which all m agents are working on some task.
- **Starved intervals**: timesteps at which at least one agent is idle because no known unresolved task is available (all tasks in $\mathcal{K}_t$ are either resolved or currently being processed by other agents).

Busy intervals. During busy intervals, the m agents collectively process tasks at rate m. The total busy time is therefore at most

$$B(\omega) \le \frac{1}{m} \sum_{s \in \mathcal{T}^*} T_s(\omega) = \frac{W(\omega)}{m}.$$

Starved intervals. During a starved interval, fewer than m tasks are available in $\mathcal{K}_t$ (net of those currently being processed). In particular, at least one agent is idle, and every task in $\mathcal{K}_t$ is either resolved, currently being processed, or not yet known. The idle agents are waiting for a currently-running task to complete and reveal new dependencies.
We claim the total starved time can be charged to the revelation chain ending at s^*. To see this, consider the sequence of tasks that "blocked" s^* from earlier completion. Let $s^* = s_0^*$. Since s^* is in $\mathcal{T}^*$, either $s^* \in \Phi^*$ (known from the start) or s^* was revealed as a dependency of some earlier task s_1^* that was completed during RLS. In the latter case, $s_1^* \to s_0^*$ in G^{rev}. Repeating, we obtain a chain $s_\ell^* \to \cdots \to s_1^* \to s_0^*$ in G^{rev} with $s_\ell^* \in \Phi^*$.
During each starved interval, the interval ends when some running task completes. If s^* was in the queue but being processed, the starved interval contributes to the processing time of s^* itself. If s^* was not yet in the queue, the starved interval is bounded by the remaining processing time of the task in the revelation chain whose completion will (eventually) reveal s^*.

In either case, the total starved time is bounded by the total processing time along the revelation chain minus the processing time of s^* itself (which is counted in the final step):

$$I(\omega) \leq \sum_{i=0}^{\ell} T_{s_i^*}(\omega) - T_{s_0^*}(\omega) \leq D(\omega) - T_{s_0^*}(\omega).$$

Combining:

$$\begin{aligned} C_{\max}^{\mathrm{RLS}}(\omega) &= B(\omega) + I(\omega) + T_{s_0^*}(\omega) \\ &\leq \frac{W(\omega)}{m} + D(\omega) - T_{s_0^*}(\omega) + T_{s_0^*}(\omega) \\ &= \frac{W(\omega)}{m} + D(\omega). \end{aligned}$$

Wait—this gives $W/m + D$, not $W/m + (1 - 1/m)D$. We refine as follows. The busy-interval bound double-counts the processing of tasks in the chain: the chain tasks contribute both to W/m (via the total work) and to the chain bound. Separating the chain from the remaining work:

$$C_{\max}^{\mathrm{RLS}}(\omega) \leq \frac{W(\omega) - D_{\mathrm{chain}}(\omega)}{m} + D_{\mathrm{chain}}(\omega) = \frac{W(\omega)}{m} + \left(1 - \frac{1}{m}\right) D_{\mathrm{chain}}(\omega),$$

where $D_{\mathrm{chain}}(\omega) = \sum_{i=0}^{\ell} T_{s_i^*}(\omega) \leq D(\omega)$.
Since $\mathrm{OPT}(\omega) \geq \max\{W(\omega)/m,\ D(\omega)\}$ (Theorem 2.4):

$$C_{\max}^{\mathrm{RLS}}(\omega) \leq \frac{W(\omega)}{m} + \left(1 - \frac{1}{m}\right) D(\omega) \leq \left(2 - \frac{1}{m}\right) \mathrm{OPT}(\omega).$$

Taking expectations completes the proof. □

Remark (Comparison to Graham's theorem). Theorem 2.5 is the MATP analogue of Graham's classical $(2 - 1/m)$-approximation for $P \mid \mathrm{prec} \mid C_{\max}$. The proof structure parallels Graham's: decompose time into busy and "starved" intervals, bound busy time by W/m, and bound starved time by the longest chain. The key difference is that Graham's "longest precedence chain" (which bounds forced-idle time due to *operational* precedence) is replaced by the serial depth (which bounds forced-idle time due to *informational* precedence—the online revelation of dependencies).

2.4.3.2 Unrelated agents, single-assignment.

When agents are heterogeneous, the problem inherits the additional complexity of the unrelated-machines model. The following is a direct consequence of the Lenstra–Shmoys–Tardos LP rounding scheme.

Proposition 2.6 (Unrelated agents, no dependencies)**.** *For a Non-Conjecturing PPI MATP instance with m unrelated agents, deterministic processing times, trivial dependencies ($\delta^* \equiv \varnothing$), and single-assignment, the problem reduces to $R\|\,C_{\max}$, which admits a 2-approximation via LP rounding (Lenstra–Shmoys–Tardos). Moreover, $R\|\,C_{\max}$ cannot be approximated within a factor of $\frac{3}{2} - \varepsilon$ for any $\varepsilon > 0$ unless* $\mathrm{P} = \mathrm{NP}$.

With nontrivial dependencies and unrelated agents, the approximation landscape is less favorable, and the interaction between agent heterogeneity, latent dependencies, and OR-redundancy remains an open direction.

2.4.3.3 OR-redundant reactive list scheduling.

When the number of known tasks is smaller than the number of agents ($|\mathcal{K}_t| < m$), single-assignment RLS leaves agents idle. The OR-redundant variant assigns excess agents to tasks already being processed by other agents, reducing expected completion times via parallel attempts.

Proposition 2.7 (Benefit of OR-redundancy)**.** *Consider a Non-Conjecturing PPI MATP with m identical agents, a single target sequent $\Phi^* = \{s_0\}$, and a revelation chain $s_0 \to s_1 \to \cdots \to s_k$ of depth $k+1$ (i.e., $\delta^*(s_i) = \{\phi_{i+1}\}$ for $i < k$ and $\delta^*(s_k) = \varnothing$), with i.i.d.* $\mathrm{Exp}(\lambda)$ *processing times across all agents and tasks. Then:*

1. *Under single-assignment RLS: $\mathbb{E}[C_{\max}] = (k+1)/\lambda$.*
2. *Under OR-redundant RLS (all m agents assigned to the current task): $\mathbb{E}[C_{\max}] = (k+1)/(m\lambda)$.*

Thus OR-redundancy yields a factor-m speedup on pure revelation chains.

Proof. Under single-assignment, each task is processed by one agent with $\mathrm{Exp}(\lambda)$ time. The $k+1$ tasks are processed serially (each is discovered only upon completion of the previous), giving total expected time $(k+1)/\lambda$.
Under OR-redundancy, all m agents work on each task. By the memoryless property, the minimum of m independent $\mathrm{Exp}(\lambda)$ variables is $\mathrm{Exp}(m\lambda)$ with mean $1/(m\lambda)$. The $k+1$ tasks still require serial processing, but each takes $1/(m\lambda)$ in expectation, giving $(k+1)/(m\lambda)$. □

Remark (OR-redundancy vs. task parallelism). Proposition 2.7 shows that OR-redundancy is maximally beneficial on pure revelation chains, where task parallelism is impossible. In contrast, when the revelation DAG is wide (many independent tasks), task parallelism dominates and OR-redundancy provides little advantage. The optimal policy must balance these two forms of parallelism based on the DAG structure and processing time distributions. Computing the optimal balance is itself NP-hard in general.

2.4.4 Summary

Restriction	Complexity	Best ratio	Reference
m identical, $\delta^* \equiv \varnothing$	str. NP-hard ($m \geq 3$)	$\frac{4}{3} - \frac{1}{3m}$ (LPT)	Graham
m identical, latent δ^*	str. NP-hard ($m \geq 3$)	$2 - \frac{1}{m}$ (RLS)	Thm. 2.5
m unrelated, $\delta^* \equiv \varnothing$	str. NP-hard	2 (LP round.)	LST
m unrelated, latent δ^*	str. NP-hard	open	—
Any, + OR-redundancy	$\leq$ std. complexity	$\leq$ std. ratio	Rem. 2.4.3.3

Table 2.1. Complexity and approximation landscape for Non-Conjecturing PPI MATP. "Latent δ^*" indicates the general case with unique but unknown dependencies. "LPT" is Longest Processing Time first; "RLS" is Reactive List Scheduling (Thm. 2.5); "LST" is Lenstra–Shmoys–Tardos LP rounding; "str." abbreviates "strongly." All approximation ratios are with respect to the expected makespan objective.

NOTE THAT WE DON'T REALLY TOUCH CONJECTURING HERE, AS IT IS ReALLY HARD TO ANALYZE? CAN WE GET ANY BOUNDS ON CONJECTURING with PPI? Even just weak to show its much harder?

Remark (Limitations and forward pointers). The Non-Conjecturing PPI MATP captures the scheduling-theoretic core of centralized multi-agent theorem proving, but several features of the full MATP lie beyond this model:

1. **Conjecturing.** Enabling conjecturing ($\mathcal{C}_0 \subsetneq \mathcal{F}$) introduces an online component: new sequents are generated during execution, and the planner must decide when to invest in exploration (conjecturing new lemmas) versus exploitation (proving known tasks). The problem becomes an instance of stochastic online scheduling with endogenous job generation.

2. **Private information.** Dropping the PPI assumption restores the POMDP structure. The planner can no longer observe agents' private proof progress, introducing an information-elicitation problem fundamentally different from the combinatorial scheduling problem studied here. This motivates the mechanism-design analysis of Section 2.4.5.

3. **Library-state-dependent proof kernels.** If the proof kernel $\mathcal{K}^{\alpha}_{\text{proof}}$ depends on the current library state (e.g., newly proven lemmas accelerate subsequent proofs), processing times become state-dependent and correlated, breaking the independence

assumptions underlying the scheduling model. This induces positive externalities between proof tasks that further complicate the optimization. **I DOn't think library state dependency is necessarily part of MATP, as we really only care about sequents, which are all more or less independent. However, i suppose as a job is complete when the sequent is fully resolved, which depends on how long its taks for its dependencies to be resolved, there is some indirect dependence on library state? idk**

ALSO WE SHOULD PROBABLY PROVE THAT AN OPTIMAL POLICY FOR A FULL MATP WITHOUT CONJECTURING IS ALSO OPTIMAL FOR THE NON-CONJECTURING PPI VERSION

2.4.5 Limits of Centralization

The centralized MATP problem provides a strong baseline: a single planner allocates theorem-proving effort across agents so as to optimize a global objective under partial observability. However, this formulation presumes a level of centralized control that is unrealistic in the motivating multiagent setting. In practice, each prover controls its own compute, possesses rich local search state that is not fully communicable, and decides whether or not to share intermediate results. Once these assumptions are relaxed, the problem is no longer one of single-agent control but of strategic interaction under partial information. This motivates a game-theoretic formulation.

Chapter 3

Decentralized Multi-Agent Theorem Proving

The centralized MATP formulation presumes a benevolent planner with the authority to assign tasks to all agents. In practice, each agent controls its own compute, maintains private proof state, and decides independently what to prove and whether to share results. We now reformulate the problem as a multi-player game, first showing that decentralization introduces externalities that lead to inefficient equilibria, and then introducing a market mechanism that internalizes these externalities and aligns individual incentives with the social objective.

3.1 Background: Multi-Agent Decision Making

We recall the standard frameworks for decentralized sequential decision-making under partial information.

Definition 3.8 (Decentralized POMDP (Dec-POMDP))**.** A discrete-time Dec-POMDP is a tuple

$$\mathcal{D} = \Big(\mathcal{N},\ \mathsf{S},\ \{\mathcal{A}^\alpha\}_{\alpha\in\mathcal{N}},\ \{\mathsf{O}^\alpha\}_{\alpha\in\mathcal{N}},\ T,\ T^0,\ \{Z^\alpha\}_{\alpha\in\mathcal{N}},\ r\Big),$$

where $\mathcal{N}$ is a finite set of agents, S is the state space, $\mathcal{A}^\alpha$ and O^α are the action and observation spaces for agent α, T and T^0 are the transition and initial-state kernels, $Z^\alpha(\cdot \mid s, a, s')$ is agent α's observation kernel, and $r : \mathsf{S}\times\mathcal{A} \to \mathbb{R}$ is a *shared* reward function (where $\mathcal{A} := \prod_\alpha \mathcal{A}^\alpha$). The defining feature is that all agents share the same reward r, making the problem *cooperative*: agents wish to maximize the same objective but must do so using only their private observation histories.

Definition 3.9 (Partially Observable Stochastic Game (POSG))**.** A POSG generalizes the Dec-POMDP by replacing the shared reward with *individual* reward functions $\{r^\alpha\}_{\alpha\in\mathcal{N}}$, where $r^\alpha : \mathsf{S}\times\mathcal{A} \to \mathbb{R}$. A Dec-POMDP is a POSG in which $r^\alpha = r^\beta$ for all α, β. When the

reward functions differ, agents have potentially conflicting objectives, and the appropriate solution concept shifts from cooperative optimality to game-theoretic equilibrium.

Each agent α selects actions according to a policy π^α that maps its private action–observation history $h_t^\alpha := (o_0^\alpha, a_0^\alpha, \ldots, o_t^\alpha)$ to a distribution over $\mathcal{A}^\alpha$. A joint policy is $\pi = (\pi^\alpha)_{\alpha \in \mathcal{N}}$, and agent α's expected discounted return under π is

$$R^\alpha(\pi) := \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \gamma^t \, r^\alpha(s_t, a_t) \right].$$

Definition 3.10 (Nash equilibrium)**.** A joint policy π^* is a *Nash equilibrium* (NE) of a POSG if no agent can improve its expected return by unilaterally deviating:

$$R^\alpha(\pi^{*\alpha}, \pi^{*,-\alpha}) \;\geq\; R^\alpha(\tilde{\pi}^\alpha, \pi^{*,-\alpha}) \qquad \text{for all } \alpha \in \mathcal{N},\ \tilde{\pi}^\alpha \in \Pi^\alpha.$$

Definition 3.11 (Social welfare and Price of Anarchy)**.** The *social welfare* of a joint policy π is $\mathrm{SW}(\pi) := \sum_{\alpha \in \mathcal{N}} R^\alpha(\pi)$. The *Price of Anarchy* (PoA) is

$$\mathrm{PoA} := \frac{\sup_\pi \mathrm{SW}(\pi)}{\inf_{\pi^* \in \mathrm{NE}} \mathrm{SW}(\pi^*)},$$

measuring the worst-case efficiency loss from strategic behavior relative to the social optimum.

3.1.1 Complexity of decentralized control.

Proposition 3.8 (Complexity of Dec-POMDP and POSG)**.**

1. *Computing an optimal joint policy for a finite-horizon Dec-POMDP is NEXP-complete (Bernstein, Givan, and Zilberstein, 2002).*

2. *Computing a Nash equilibrium of a finite POSG is at least PPAD-hard (by reduction from Nash equilibrium computation in normal-form games).*

Thus, even the cooperative problem (Dec-POMDP) is vastly harder than the centralized problem (POMDP, which is PSPACE-complete). Decentralization incurs a fundamental computational cost.

3.2 The Decentralized MATP Game

We now formulate the *Decentralized MATP* as a POSG by distributing the centralized planner's authority to the individual agents.

Definition 3.12 (Decentralized MATP POSG)**.** Given the same primitives as the centralized MATP (agent set $\mathcal{N}$, formula space $\mathcal{F}$, proof and conjecture kernels, library structure), the *Decentralized MATP* is the POSG

$$\mathcal{G}_{\text{MATP}} = \Big(\mathcal{N},\ \mathsf{S},\ \{\mathcal{A}^\alpha\}_{\alpha\in\mathcal{N}},\ \{\mathsf{O}^\alpha\}_{\alpha\in\mathcal{N}},\ T,\ T^0,\ \{Z^\alpha\}_{\alpha\in\mathcal{N}},\ \{r^\alpha\}_{\alpha\in\mathcal{N}},\ \gamma\Big)$$

where:

- **State space** S is identical to the centralized MATP (private libraries, public library, job descriptors, query models, cumulative effort).
- **Action spaces** $\mathcal{A}^\alpha$ are identical to the per-agent action spaces of the centralized MATP: each agent independently selects Prove, Conj, Pub, Qry, or $\perp$.
- **Observations** O^α are agent-specific: agent α observes its own private library $\mathcal{L}_t^\alpha$, its own portfolio/balance, its own query responses Ω_t^α, and the public library $\mathcal{L}_t^0$. Crucially, agent α does *not* observe other agents' private libraries, job statuses, or cumulative effort maps.
- **Transition kernel** T is identical to the centralized MATP (the physics of proving, conjecturing, and publishing are unchanged).
- **Individual rewards** r^α encode each agent's private objective. We defer the precise specification of r^α to the discussion below, noting that the choice of reward structure is the central design variable.

The key structural change from the centralized MATP is that there is no planner: each agent α selects its own action a_t^α based on its own observation history h_t^α. The joint action $a_t = (a_t^\alpha)_{\alpha\in\mathcal{N}}$ emerges from the independent, simultaneous choices of all agents.

3.2.1 Cooperative baseline: Dec-POMDP.

As a baseline, one may set $r^\alpha = r$ for all α, recovering a Dec-POMDP in which all agents cooperatively maximize the centralized MATP objective (e.g., minimizing the time to fully resolve Φ^*). In this case, communication occurs solely through publication: an agent that publishes a resolved sequent effectively "communicates" its result to all others via the shared library. The Dec-POMDP formulation captures the difficulty of coordination under communication constraints, but assumes agents are willing to cooperate—they share the social objective and have no private interests.

Remark (Communication via publication). In the Dec-POMDP, the only communication channel between agents is the shared library $\mathcal{L}_t^0$. An agent's publication action $(\text{Pub}, (C, R))$

is the sole mechanism by which private information enters the public domain. This is a severely constrained communication model: agents cannot send arbitrary messages, and publication is "all-or-nothing" (either a result is in the shared library or it is not). The Dec-POMDP's NEXP-completeness (Proposition 3.8) reflects, in part, the difficulty of coordinating under such constraints.

3.2.2 Individual rewards and the externality problem.

In practice, agents are not purely cooperative. Each agent has its own research agenda, computational resources, and incentives. A more realistic model assigns each agent an individual reward that depends on the agent's own contributions and on the state of the shared library. Consider a natural class of *credit-based* reward functions: agent α's reward is determined by the sequents that α has proven and that are credited to α in the metadata Θ_t^0.

Definition 3.13 (Credit-based reward)**.** Fix a target valuation $v : \mathcal{S} \to \mathbb{R}_{\geq 0}$ assigning value to fully resolved sequents. Agent α's credit-based per-step reward is

$$r^{\alpha}_{\text{credit}}(s_t, a_t) := \sum_{s \in \mathcal{FRS}^0_{t+1} \setminus \mathcal{FRS}^0_t} v(s) \cdot \mathbf{1}[\Theta^0_{t+1}(s) = (\cdot, \alpha)],$$

i.e., agent α is rewarded for each newly fully resolved sequent in the shared library that is credited to α.

Under credit-based rewards, agents have incentive to prove valuable sequents and receive credit. However, this reward structure introduces a fundamental externality.

3.3 The Publication Externality

We now show that the decentralized MATP with credit-based rewards exhibits a *publication externality*: an agent's publication of a resolved sequent creates social value (by enabling other agents' proofs) but generates no direct reward for the publisher.

Definition 3.14 (Publication externality)**.** A *publication externality* exists at state s_t if there exists an agent α and a publication action $a^\alpha = (\text{Pub}, (C, R))$ such that:

1. Publishing strictly increases some other agent β's expected future reward:

$$\mathbb{E}[R^{\beta}_{\geq t+1} \mid s_t,\ a^\alpha = (\text{Pub}, (C, R))] > \mathbb{E}[R^{\beta}_{\geq t+1} \mid s_t,\ a^\alpha = \perp],$$

   where $R^{\beta}_{\geq t+1} := \sum_{\tau=t+1}^{\infty} \gamma^{\tau-t-1} r^{\beta}(s_\tau, a_\tau)$.

2. Publishing does not increase α's own expected future reward:

$$\mathbb{E}[R^{\alpha}_{\geq t+1} \mid s_t,\ a^{\alpha} = (\text{Pub}, (C, R))] \leq \mathbb{E}[R^{\alpha}_{\geq t+1} \mid s_t,\ a^{\alpha} = \perp].$$

Theorem 3.9 (Existence of publication externalities)**.** *In any Decentralized MATP POSG with $|\mathcal{N}| \geq 2$, credit-based rewards, and non-trivial latent dependencies between agents' targets, publication externalities exist.*

Proof. We construct a witnessing instance. Let $\mathcal{N} = \{\alpha, \beta\}$. Let the target sets be $\Phi^*_\alpha = \{s_\alpha\}$ and $\Phi^*_\beta = \{s_\beta\}$, with $v(s_\alpha) = v(s_\beta) = 1$ and $v(s) = 0$ otherwise. Suppose the latent dependency structure satisfies $\delta^*(s_\beta) = \{\phi_\alpha\}$, where ϕ_α is the consequent of s_α. Thus s_β cannot be fully resolved until s_α is also resolved in the shared library.

At some time t, suppose α has privately resolved s_α (i.e., $s_\alpha \in \mathcal{RS}^\alpha_t$) but has not published it ($s_\alpha \notin \mathcal{RS}^0_t$). Agent α's own target s_α is already resolved in α's private library, so α's credit-based reward for s_α is earned only when s_α enters $\mathcal{FRS}^0_t$—which requires publication. Under credit-based rewards, α receives $v(s_\alpha) = 1$ upon publishing s_α (since $\delta^*(s_\alpha) = \varnothing$ makes it immediately fully resolved upon publication). So publishing actually does help α in this case.

To create a pure externality, modify the instance so that α's target is s'_α with $\delta^*(s'_\alpha) = \varnothing$ and $s'_\alpha \neq s_\alpha$, while α has also privately resolved s_α as a byproduct of working toward s'_α. Now α receives credit for s'_α (its own target) but not for s_α (which is not in α's target set and has $v(s_\alpha) = 0$ from α's perspective). Publishing s_α costs α one timestep but provides α no reward (since $v(s_\alpha) = 0$ for α). However, publishing s_α enables β to fully resolve s_β (since s_β's dependency on ϕ_α is satisfied). Thus:

- α's reward from publishing s_α: 0 (no credit, cost of one timestep).
- β's reward from α publishing s_α: $v(s_\beta) = 1$ (enables full resolution of β's target).

This satisfies Definition 3.14. □

Corollary 3.9.1 (Free riding and underinvestment)**.** *Under credit-based rewards, the unique Nash equilibrium of the externality instance in Theorem 3.9 has α not publishing s_α, leading to β failing to fully resolve s_β. The social welfare at this NE is strictly lower than the social optimum (in which α publishes s_α, enabling β to resolve s_β).*

The publication externality is the core market failure in decentralized theorem proving: agents who produce useful intermediate results have no incentive to share them. This leads to duplication of effort (multiple agents independently proving the same lemma), underinvestment in "infrastructure" lemmas (results useful to many but credited to none), and withholding of results (agents hoarding private proofs to maintain competitive advantage).

Remark (Beyond credit-based rewards). The publication externality is not an artifact of the credit-based reward structure. Any individual reward function that fails to compensate agents for the social value of their publications will exhibit a similar externality. The fundamental issue is that publication creates positive externalities—benefits to other agents that are not captured by the publisher's reward. Eliminating this externality requires a mechanism that transfers value from beneficiaries to publishers: a market.

3.4 Market-Mediated MATP

We now introduce a market mechanism that internalizes the publication externality by enabling agents to trade state-contingent contracts whose settlement is triggered by publicly verifiable events (sequent resolution in the shared library).

3.4.1 Collateralized bilateral contract market.

We equip the Decentralized MATP POSG with a *bilateral contract market* operating as follows.

A *contract type* is a triple $\chi = (s, T, \ell) \in \mathcal{CS}_0 \times \mathbb{N} \times [0, 1]$, representing a normalized unit-notional claim on the event that sequent s becomes publicly resolved by deadline T, with maximum loss parameter ℓ. Each contract type induces two complementary positions, the *long* side χ^+ and the *short* side χ^-, with terminal payoffs:

$$\Pi(\chi^+) = \begin{cases} 1 - \ell & \text{if } s \in \mathcal{RS}_T^0, \\ -\ell & \text{otherwise,} \end{cases} \qquad \Pi(\chi^-) = -\Pi(\chi^+).$$

Contracts are traded via *public offers* $o = (\chi, \sigma, q, p)$, where $\sigma \in \{+, -\}$ is the side offered, $q \in \mathbb{N}^+$ is quantity, and $p \in [0, 1]$ is the per-unit price. When agent β accepts quantity $q' \leq q$ from an offer posted by α, the position $q'\chi^\sigma$ transfers to β's holdings and cash $q'p$ transfers from β to α.

Contracts are always issued as matched pairs $(q\chi^+, q\chi^-)$, ensuring zero net creation of wealth.

A *collateral constraint* requires that every agent's portfolio satisfies $\underline{B}^\alpha \geq 0$ at all times, where $\underline{B}^\alpha := c^\alpha - \sum_{x \in H^\alpha \cup O^\alpha} \sup_\omega(-\Pi_\omega(x))_+$ is the minimum marked balance (cash minus worst-case liabilities). This ensures no agent can default.

At each timestep t, all contracts with deadline $T = t$ are *settled*: payoffs are applied to holdings based on the public library state $\mathcal{RS}_t^0$, and settled contracts are removed.

3.4.2 Market-augmented reward.

In the market-mediated MATP, each agent α's reward is the change in portfolio balance:

$$r^{\alpha}_{\text{mkt}}(s_t, a_t) := B^{\alpha}_{t+1} - B^{\alpha}_t,$$

where $B^{\alpha}_t = \beta(\mathcal{P}^{\alpha}_t)$ is the canonical balance of α's portfolio (cash plus mark-to-market value of positions). This reward naturally captures trading profits, proof costs, and contract settlement payoffs.

Definition 3.15 (Market-Mediated MATP (VAMP))**.** The *Verifiable Allocation Market POSG* (VAMP) is the POSG obtained by augmenting the Decentralized MATP with the bilateral contract market:

$$\mathcal{G}_{\text{VAMP}} = \Big(\mathcal{N},\ \mathcal{W},\ \{\mathcal{A}^{\alpha}\}_{\alpha\in\mathcal{N}},\ \{\mathsf{O}^{\alpha}\}_{\alpha\in\mathcal{N}},\ \mathcal{T},\ \{r^{\alpha}_{\text{mkt}}\}_{\alpha\in\mathcal{N}},\ \gamma,\ b_0,\ \mathscr{M}\Big),$$

where the world state $\mathcal{W}$ includes the market state $\mathcal{M}_t = (\{\mathcal{P}^{\alpha}_t\}_{\alpha}, \mathcal{ML}_t)$, agent actions include market actions (posting offers, accepting offers) alongside proof/conjecture/publication actions, and the transition kernel incorporates the two-stage construction (non-market transitions followed by market transitions) as specified by the market mechanism $\mathscr{M}$. The full formal definition is given in Chapter **??**.

3.5 Internalization of Externalities

We now show that the bilateral contract market internalizes the publication externality identified in Theorem 3.9.

Theorem 3.10 (Market internalization of publication externalities)**.** *Consider the VAMP game $\mathcal{G}_{\text{VAMP}}$ with the bilateral contract market. Suppose agent β values the public resolution of sequent s by time T at $v_\beta > 0$ (in the sense that β's expected future reward increases by v_β if $s \in \mathcal{RS}^0_T$). Then:*

1. ***Demand expression.*** *Agent β can express demand for the resolution of s by purchasing long contracts $\chi^+ = (s, T, \ell)^+$ at a price $p < v_\beta$. This is individually rational for β whenever the market price is below β's valuation.*

2. ***Proof incentive.*** *If agent α can resolve s at expected cost c_α, then α can profitably buy long contracts at any price $p < 1 - \ell - c_\alpha$, then prove and publish s, collecting the settlement payoff $1 - \ell$ per contract. The publication externality is internalized: α's reward from proving and publishing now includes the contract profit $(1-\ell)-p-c_\alpha > 0$.*

3. ***Value transfer.*** *In any trade, the contract price p transfers value from the buyer (who benefits from resolution) to the seller of the complementary side. At equilibrium, the long price p^* satisfies*

$$c_{\min} \;\leq\; 1 - \ell - p^* \;\leq\; v_{\max},$$

where $c_{\min} = \min_\alpha c_\alpha$ is the cheapest prover's cost and $v_{\max} = \max_\beta v_\beta$ is the highest valuation. Trades occur whenever there are gains from trade ($c_{\min} < v_{\max}$), and the market price reflects the marginal cost of proof provision.

Proof sketch. (1) Agent β buys χ^+ at price p. If s is resolved by T, β receives $1-\ell$ from settlement. β's expected profit from the contract is $\Pr(s \in \mathcal{RS}_T^0)(1-\ell) - \ell \cdot \Pr(s \notin \mathcal{RS}_T^0) - p$. If β's valuation v_β exceeds p, β has positive expected surplus from purchasing.
(2) Agent α buys χ^+ at price p, then proves s at cost c_α and publishes. Settlement yields $1-\ell$ per contract. α's net payoff is $(1-\ell) - p - c_\alpha$. This is positive whenever $p < 1 - \ell - c_\alpha$. Crucially, α now has a direct financial incentive to both prove *and* publish s: the settlement payoff is triggered by public resolution ($s \in \mathcal{RS}_T^0$), which requires publication.
(3) At equilibrium, the long price must make the marginal buyer indifferent ($p^* \approx v_{\text{marginal}} - (1-\ell)$, adjusted for resolution probability) and the marginal prover indifferent ($p^* \approx (1-\ell) - c_{\text{marginal}}$). Trades occur when these ranges overlap, i.e., when some prover's cost is below some buyer's valuation. □

Remark (Comparison to the externality instance). Revisiting the instance from Theorem 3.9: without a market, α has no incentive to publish s_α (zero reward, positive cost). With the market, β can buy long contracts on s_α, creating a financial incentive for any agent (including α) to prove and publish s_α. The market price reflects β's valuation, and the resulting trade transfers value from β (beneficiary) to the prover (provider), eliminating the externality.

Remark (Collateralization and manipulation). The collateral constraint $\underline{B}^\alpha \geq 0$ ensures that agents cannot take on unbounded liabilities, preventing default risk. This is crucial for incentive alignment: without collateralization, agents could sell short contracts on sequents they intend to prevent from being resolved (a form of market manipulation). Full collateralization bounds the worst-case loss for every agent, ensuring that all positions are credibly backed. However, collateralization also constrains participation: agents with limited capital cannot take large positions, potentially limiting market liquidity. The tradeoff between manipulation-resistance and liquidity is a standard theme in market design.

3.6 Equilibrium Analysis

We now analyze the game-theoretic properties of the VAMP game.

3.6.1 Existence of equilibria.

Proposition 3.11 (Existence of Nash equilibria)**.** *If the VAMP game $\mathcal{G}_{\text{VAMP}}$ has finite state, action, and observation spaces, compact strategy sets, and continuous payoff functions (in the policy topology), then a Nash equilibrium exists in mixed (behavioral) strategies.*

Proof sketch. By Glicksberg's theorem, every finite game with compact strategy sets and continuous payoffs has a Nash equilibrium in mixed strategies. The VAMP's finite-horizon truncation satisfies these conditions. For the infinite-horizon discounted game, existence follows from the standard fixed-point argument on the space of behavioral strategies, applying Kakutani's theorem to the best-response correspondence. □

Remark (Markov perfect equilibria)*.* For computational and analytical tractability, one often restricts to *Markov perfect equilibria* (MPE), in which each agent's policy depends on the current state only (not the full history). In the VAMP, a state-dependent policy $\pi^\alpha(s_t)$ conditions on agent α's current observation of the world state (private library, portfolio, public library). Under standard regularity conditions, MPE exist and are refinements of Nash equilibria.

3.6.2 Price of Anarchy.

The Price of Anarchy measures the efficiency loss from strategic behavior relative to the social optimum. In our setting, the social optimum corresponds to the centralized MATP solution (a single planner maximizing social welfare), and the PoA compares the worst NE of the VAMP game against this benchmark.

Proposition 3.12 (PoA lower bound)**.** *The Price of Anarchy of the Decentralized MATP POSG* without *a market (credit-based rewards) is unbounded: for any $M > 0$, there exists an instance with* PoA $> M$*.*

Proof. Consider the externality instance from Theorem 3.9 with n agents, where each agent α_i privately resolves a sequent s_i that is a dependency of the next agent α_{i+1}'s target. Without a market, no agent has incentive to publish, so only agent α_1's target is achievable in the worst NE. The social optimum achieves all n targets. Thus PoA $\geq n$, which is unbounded as $n \to \infty$. □

Proposition 3.13 (PoA with market)**.** *In the VAMP game with the bilateral contract market and sufficient initial liquidity, the Price of Anarchy is bounded. Specifically, if all agents have sufficient capital to participate in the market (i.e., initial balances $c_0^\alpha \geq \bar{c}$ for some threshold $\bar{c}$ depending on the instance), then at any Nash equilibrium, every sequent s with social value*

exceeding the cheapest resolution cost is resolved. In particular, the PoA is at most a constant depending on the market parameters (ℓ, γ).

Proof sketch. At any NE, if a sequent s has positive social surplus (some agent values its resolution above the cheapest prover's cost), then a profitable deviation exists for some agent: buy long, prove, publish, collect settlement. This deviation is unilateral and profitable, contradicting NE if s remains unresolved. The bound on PoA follows from bounding the fraction of social value that is captured by market frictions (transaction costs implicit in the bid-ask spread, the loss parameter ℓ, and discounting). □

3.6.3 Computational complexity of equilibria.

Proposition 3.14 (Complexity of VAMP equilibria)**.**

1. *Computing an exact Nash equilibrium of the VAMP game is PPAD-hard (by reduction from bimatrix games, since the market introduces a general-sum structure even with two agents).*

2. *Computing an optimal Dec-POMDP solution (the cooperative benchmark) is NEXP-complete.*

3. *The VAMP's state space, augmented with market state (portfolios, order book), is exponentially larger than the centralized MATP state space. Exact equilibrium computation is therefore doubly intractable: both the game-theoretic and the state-space dimensions are hard.*

3.7 Intractibility Consequences

These intractability results motivate *learning-based approaches* to approximating equilibria in the VAMP game. Rather than computing exact Nash equilibria (which is provably hard), one may deploy multi-agent reinforcement learning (MARL) algorithms that converge to approximate equilibria through self-play. The VAMP's verifiable settlement mechanism (public resolution of sequents triggers deterministic payoffs) provides a clean reward signal for learning, and the market's price signals serve as a coordination mechanism that reduces the effective dimensionality of the strategy space. We develop this approach in the following chapter, where we instantiate the VAMP framework with specific MARL algorithms (inner-loop MAPPO self-play, outer-loop population-based training approximating PSRO) applied to a synthetic theorem-proving market.

Chapter 4

Multiagent Reinforcement Learning for VAMPs

Chapter 5

Experiments

TODO PASTE FROM OVERLEAF WRITEUP

Chapter 6

Application to Neural Theorem Proving

TODO PASTE FROM OVERLEAF WRITEUP

Chapter 7

Conclusion

Bibliography